Roteiro do vídeo-relatório — revisão com medições reais do CORE

Orientações iniciais ao grupo — fora dos slides e da narração

Documentação, evidências e limites atuais

A documentação interna e o histórico de desenvolvimento serviram para organizar o trabalho. Há trechos desatualizados e propostas que não representam validações executadas. **Esses documentos não fundamentam a análise final dos resultados**: a análise usa as medições, suas condições de execução e os cálculos sobre elas. As observações sobre documentação ficam exclusivamente nesta abertura.

A nova pasta comandos/ acrescenta 12 saídas reais fornecidas de ping, traceroute e iperf, nos dois sentidos e nos dois perfis. A comparação detalhada relaciona essas saídas à topologia, à campanha e às simulações. A ausência de logs de calibração apontada anteriormente foi parcialmente resolvida: agora há conectividade, rotas e taxas observadas. Não é correto continuar dizendo que a calibração é inteiramente hipotética.

A pasta `comandosip/comandosip` é tratada como A e `comandosipB/comandosipB` como B, em coerência com nomes, taxas e RTTs. Cada combinação perfil/sentido possui um ensaio iperf de cerca de 20 s, dez pings e um traceroute. Não há datas ou identificadores que liguem essas saídas a cada rodada da campanha. Não apresentar a calibração como repetida 11 vezes ou contemporânea a todas as transferências. O arquivo `iperfR_server.txt` mostra o cliente em modo reverso, com conteúdo servidor→cliente; seu nome não muda o sentido efetivamente indicado na saída.

Para a apresentação, usar a linha **receiver** do iperf: A=93,9/94,4 Mbit/s e B=18,9/19,0 Mbit/s, respectivamente cliente→servidor e servidor→cliente. Os 40 pings receberam resposta; isso não significa ausência de retransmissões TCP, que aparecem nos quatro ensaios: 48/62 em A e 11/12 em B. Não atribuir essas retransmissões a uma causa específica nem convertê-las diretamente em percentual de perda.

Permanecem pendentes a janela comum de cada rodada da aplicação, hash do destino PUT, monitoração dos recursos, vínculo de versões/sessão e controle de cache. Os 1.804 PUTs da campanha retornam 201; a campanha mostra criação, não comprova sobrescrita. A inspeção do servidor também identificou HEAD anunciando comprimento zero e caminhos de erro PUT que merecem revisão própria. Não demonstrar tamanho remoto correto por HEAD nem declarar todas as falhas e sobrescritas validadas.

As simulações foram comparadas e permanecem separadas. Embora a calibração idealizada tenha valores próximos, a simulação da aplicação subestima a taxa do GET com um cliente: 69,1 contra 87,3 Mbit/s em A e 13,9 contra 17,7 em B, para 10 MiB. **Nenhum número sintético ou platô artificial entra nos resultados deste roteiro.** Não há necessidade de considerações fictícias para apresentar o conteúdo real disponível.

O que falta para completar o registro das medições

1. Associar cada calibração a um ID de sessão/campanha, data, revisão do código, versões, XML, endereços, rotas e parâmetros efetivos de `tc`. Preservar os logs novos, sem sobrescrevê-los. Repetir calibrações independentes se a intenção for estimar estabilidade e IC; intervalos de um segundo do mesmo iperf não são repetições independentes.

2. Registrar comando completo, direção, duração e eventual aquecimento. Usar taxa recebida do iperf, RTT de ping e atraso configurado com suas definições distintas. Realizar calibração e transferência da aplicação em momentos separados para não criar tráfego concorrente acidental.
3. Instrumentar o coordenador para registrar liberação conjunta e fim da rodada em um relógio monotônico comum, incluindo desvio de lançamento, timeouts e falhas. O `started_utc` atual é preenchido após o cliente terminar e não serve como início da tentativa. Não subtrair relógios monotônicos de hosts distintos.
4. Verificar tamanho e SHA-256 de origem/destino de GET e PUT fora da janela de desempenho. Para sobrescrita, preparar destino preexistente e conferir sua substituição; separar esse modo de carga da criação de arquivo novo.
5. Registrar recursos do host, cache, persistência e ordem das células. Calcular goodput agregado por bytes confirmados/janela comum, preservando falhas e rodadas incompletas. Usar rodada como unidade de IC e não somar taxas individuais.
6. Executar os testes funcionais pendentes e ampliar perfis mantendo um fator fixo caso se queira separar efeitos de banda e atraso. Os novos logs não resolvem essas questões por si só.

Montagem e ensaio

- **Duração:** 11 slides em **9min40s**, com 20 s de margem. A abertura técnica não é narrada. Reservar cerca de 10 s para demonstração no slide 5 e ensaiar as transições.
- **Contribuições:** a divisão entre `[Nome 1]` e `[Nome 2]` continua sendo proposta realista para dois integrantes. Confirmar e ajustar à participação efetiva antes de narrar em primeira pessoa. A autoavaliação também é uma sugestão.
- **Novo visual:** calibracao_core.png, 2560×1440, e SVG editável, para o slide 4. As tabelas extraídas preservam números exatos e sentidos. O gráfico arredonda os rótulos para uma casa decimal; a tabela do roteiro conserva três casas no RTT.
- **Reaproveitamento:** topologia e arquitetura dos slides antigos 2/3; gráficos da aplicação dos slides 6/7/8. A matriz antes no slide 4 passa ao 5. Substituir o antigo slide 9 por comparação entre iperf e aplicação; não usar a curva sintética de saturação. Os caminhos de montagem ficam apenas nesta abertura.
- **Versões:** esta revisão atualiza o roteiro Markdown e gera o visual novo de calibração; o PDF existente e os blocos antigos de slides no notebook não foram regenerados. Não usar automaticamente instruções antigas de slide 4/5/9. Médias da campanha foram recalculadas e conferidas contra os dados brutos.
- **Demonstração:** mostrar uma tentativa real já registrada, ou gravar uma execução nova identificada. Se não houver clipe, exibir os campos de uma tentativa existente e narrar sua leitura. Não apresentar animação como captura real. Localhost, se usado, deve ser identificado como demonstração funcional.

Sequência e tempo

Slide	Intervalo	Tema	Apresentador sugerido
1	0:00–0:30	Problema e objetivo	Integrante 1
2	0:30–1:20	Aplicação e decisões de implementação	Integrante 1
3	1:20–2:00	Topologia e perfis	Integrante 2
4	2:00–2:55	Rede medida nos dois sentidos	Integrante 2

Slide	Intervalo	Tema	Apresentador sugerido
5	2:55–3:45	Matriz, método e demonstração	Integrante 1
6	3:45–4:40	Tempo de resposta	Integrante 2
7	4:40–5:35	Vazão útil individual	Integrante 1
8	5:35–6:20	GET, PUT e HEAD	Integrante 1
9	6:20–7:20	Rede, aplicação e concorrência	Integrante 2
10	7:20–8:15	Contribuições individuais	Ambos
11	8:15–9:40	Autoavaliação e conclusão	Ambos

Slide 1 — Problema e objetivo

Adicionar ao slide: título “Transferência concorrente de arquivos em redes emuladas”; nomes, disciplina e pergunta “Como rede, tamanho e concorrência afetam tempo e vazão?”. Desenho simples de cliente e servidor, com exemplos de foto e vídeo.

Fala — Integrante 1:

> Desenvolvemos uma aplicação cliente-servidor de arquivos em C e avaliamos seu desempenho no CORE. O objetivo foi entender como as condições de rede, o tamanho do arquivo e a concorrência afetam tempo de resposta e vazão útil. Escolhemos cargas associadas a fotos e vídeos curtos porque representam transferências comuns em que vários usuários compartilham os mesmos recursos.

Slide 2 — Aplicação e decisões de implementação

Adicionar ao slide: diagrama `Cliente em C ⇔ TCP / subconjunto de HTTP ⇔ Servidor em C`; cartões “GET: baixar”, “PUT: enviar”, “HEAD: consulta sem corpo”; destaques “thread por conexão”, “leituras e escritas parciais” e “temporário exclusivo → fsync → rename”. Usar uma animação simples para explicar o fluxo, sem tratar a animação como execução real.

Fala — Integrante 1:

> Cliente e servidor usam sockets POSIX em C, com TCP e um subconjunto controlado de HTTP. Cada conexão atende uma operação: GET baixa o arquivo, PUT envia conteúdo e HEAD faz uma consulta sem transferir o corpo. O servidor cria uma thread por conexão, permitindo atendimentos simultâneos. Como TCP entrega um fluxo de bytes, tratamos leituras e escritas parciais. No PUT, os dados são recebidos em um temporário exclusivo; o servidor sincroniza e publica o arquivo antes de confirmar o sucesso. Essas escolhas permitem organizar a concorrência e evitam publicar um arquivo parcialmente recebido como se estivesse completo.

Slide 3 — Topologia e perfis

Adicionar ao slide: seis nós, sem desenhar o gargalo como um nó extra: `Cliente → Acesso cliente → Núcleo 1 → Núcleo 2 → Acesso servidor → Servidor`. Destacar o enlace Núcleo 1↔Núcleo 2. Endereços finais `10.10.1.10` e `10.10.5.10`. Tabela:

Perfil	Capacidade nominal	Atraso por sentido	Perda artificial configurada
A	100 Mbit/s	5 ms	0%
B	20 Mbit/s	20 ms	0%

Rodapé: “Traceroute: quatro roteadores + destino; elevação do RTT após cruzar o núcleo”.

Fala — Integrante 2:

> No CORE, cliente e servidor ficam em redes de acesso distintas, ligadas por quatro roteadores. O gargalo está entre os dois roteadores de núcleo. Em A configuramos 100 megabits por segundo e 5 milissegundos por sentido; em B, 20 megabits e 20 milissegundos. São representações simplificadas de condições de acesso fixo e móvel, sem perda artificial configurada. Os traceroutes mostram cinco saltos até o destino nos dois sentidos. A elevação do RTT aparece após a travessia do núcleo, coerente com a posição escolhida para o gargalo.

Slide 4 — Rede medida nos dois sentidos

Adicionar ao slide: gráfico com dois painéis, taxa recebida no iperf e RTT médio no ping. Alternativamente, usar a tabela abaixo como elemento principal, evitando duplicar tabela e gráfico na mesma tela:

Perfil	Sentido	iperf recebido (Mbit/s)	RTT médio (ms)
A	Cliente → servidor	93,9	10,555
A	Servidor → cliente	94,4	10,655
B	Cliente → servidor	18,9	40,588
B	Servidor → cliente	19,0	40,545

Rodapé: “Por perfil/sentido: 1 ensaio iperf ≈20 s e 10 pings. Sem perdas nos 40 pings; houve retransmissões TCP”. Não inserir barras de IC de ensaios independentes.

Fala — Integrante 2:

> Medimos a rede nos dois sentidos. No iperf, a taxa recebida ficou entre 93,9 e 94,4 megabits por segundo no perfil A, e entre 18,9 e 19 no B. Os RTTs médios ficaram próximos de 10,6 e 40,6 milissegundos, coerentes com a contribuição de ida e volta do gargalo. Cada combinação tem um ensaio de aproximadamente vinte segundos e dez pings. Todos os quarenta pings receberam resposta, mas o iperf registrou retransmissões TCP. Portanto, não confundimos ausência de perda nessa amostra de ping com ausência de retransmissões. Essas medições dão uma referência real da rede para interpretar as transferências da aplicação.

Slide 5 — Matriz, método e demonstração

Adicionar ao slide: expressão `2 perfis × 3 operações × 2 tamanhos × 3 concorrências = 36 configurações`; tamanhos 2/10 MiB; concorrências 1/8/32; 11 rodadas por configuração; cartões “396 rodadas” e “5.412 requisições”. Destaque “IC95% de Student entre rodadas”. Reservar cerca de 10 s para mostrar uma tentativa real de GET ou PUT e seus campos status, bytes, duração e taxa; não exigir digitação ao vivo.

Fala — Integrante 1:

> Combinamos dois perfis, três operações, arquivos de 2 e 10 mebibytes e concorrências de 1, 8 e 32 clientes. Foram onze rodadas em cada uma das 36 configurações, totalizando 396 rodadas e 5.412 requisições. Nesta tentativa, vemos os campos usados para acompanhar status, bytes, duração e taxa individual.

Ação: exibir a tentativa por aproximadamente 10 s.

Continuação — Integrante 1:

> A análise agrega os clientes de cada rodada e calcula o intervalo de confiança entre as onze rodadas. Clientes simultâneos compartilham recursos e não são contados como repetições independentes. HEAD fica sem vazão de conteúdo. A escolha de onze rodadas por configuração também permite descrever a variação entre execuções, em vez de depender de uma única medição. Em cada rodada concorrente, os clientes compartilham a mesma condição de rede; por isso, primeiro resumimos os clientes da rodada e depois comparamos as rodadas independentes.

Slide 6 — Tempo de resposta

Adicionar ao slide: gráfico de GET com painéis A/B, linhas de 2/10 MiB e IC95% entre rodadas. Eixo de tempo em segundos. Destaque para GET de 10 MiB:

Clientes concorrentes	Perfil A (s)	Perfil B (s)
1	0,961	4,730
32	28,105	137,936

Legenda: “Média das médias de rodada; IC95% de Student; n=11 por configuração”. Se houver pouco espaço, usar dois pares de cartões numéricos em vez da tabela.

Fala — Integrante 2:

> O tempo de resposta cresce com a concorrência, principalmente no arquivo maior. No GET de dez mebibytes com um cliente, a média foi de 0,961 segundo em A e 4,730 segundos em B. Com 32 clientes, passou para 28,105 segundos em A e 137,936 em B. O perfil B apresentou maior tempo nas dezoito comparações equivalentes de operação, tamanho e concorrência. As barras mostram a incerteza entre rodadas. A rede medida ajuda a contextualizar esse comportamento: B combina menor taxa disponível e maior RTT. Como alteramos capacidade e atraso juntos, o resultado representa o efeito combinado dessas condições, sem separar a contribuição de cada fator. A diferença é consistente nas dezoito combinações de método, tamanho e concorrência, o que torna a tendência clara dentro desta matriz. Ainda assim, as barras de confiança descrevem variabilidade entre rodadas nas células; elas não removem limitações de coleta nem transformam a comparação entre perfis em um experimento que isola a banda da latência.

Slide 7 — Vazão útil individual

Adicionar ao slide: gráfico GET A/B de vazão individual, separado por tamanho; fórmula `bytes úteis × 8 / duração_s / 10⁶`. Tabela ou cartões para GET de 10 MiB:

Clientes concorrentes	Perfil A (Mbit/s)	Perfil B (Mbit/s)
1	87,322	17,744
32	2,997	0,609

Legenda: “Taxa média por transferência; IC95% entre rodadas”.

Fala — Integrante 1:

> A vazão útil individual representa a taxa de cada transferência completa. No GET de dez mebibytes com um cliente, obtivemos aproximadamente 87,3 megabits por segundo em A e 17,7 em B. Com 32 clientes, a média individual caiu para cerca de 3 megabits em A e 0,61 em B. Nas doze comparações equivalentes de GET e PUT, B apresentou menor vazão individual. A queda por cliente é compatível com o compartilhamento dos recursos. Isso descreve a experiência de cada usuário; a taxa total do serviço exige uma medição diferente, que considere todos os bytes concluídos dentro de uma mesma janela de tempo. Por exemplo, a taxa de um cliente não pode ser multiplicada simplesmente por 32: clientes podem iniciar e terminar em instantes diferentes, e as durações individuais não compartilham necessariamente o mesmo denominador. A vazão agregada precisa usar os bytes efetivamente concluídos e o intervalo entre a liberação conjunta e o último término.

Slide 8 — GET, PUT e HEAD

Adicionar ao slide: cartões de sentido “GET: servidor→cliente”, “PUT: cliente→servidor” e “HEAD: sem conteúdo”. Tabela para um cliente:

Operação e tamanho de referência	Tempo A (ms)	Tempo B (ms)
GET, 10 MiB	960,8	4.729,8
PUT, 10 MiB	1.171,9	4.814,3
HEAD, referência de 2 MiB	22,1	83,2

Nota curta: “PUT da campanha: criação de arquivos; HEAD: vazão de conteúdo não aplicável”.

Fala — Integrante 1:

> GET e PUT movimentam conteúdo em sentidos opostos e têm custos diferentes. Com um cliente e dez mebibytes, o PUT levou cerca de 1,17 segundo em A e 4,81 em B. Nesta campanha, os PUTs retornaram status 201, correspondente à criação de arquivos. O HEAD sobre a referência de dois mebibytes levou cerca de 22 e 83 milissegundos. Como não transfere o corpo, sua vazão de conteúdo não se aplica. As diferenças de GET e PUT podem envolver conexão, leitura, escrita e confirmação; os tempos sozinhos não isolam qual desses custos explica cada diferença.

Slide 9 — Rede, aplicação e concorrência

Adicionar ao slide: comparação de GET de 10 MiB com um cliente e iperf no mesmo sentido servidor→cliente:

Perfil	iperf recebido (Mbit/s)	GET individual (Mbit/s)
A	94,4	87,322
B	19,0	17,744

Usar barras agrupadas com eixo começando em zero ou manter a tabela. Rodapé: “Ensaios distintos; iperf ≈20 s; GET inclui a operação completa”. Ao lado, desenho de clientes compartilhando o enlace, com a frase “Taxa individual ≠ taxa agregada”. Não inserir curva de saturação com pontos inventados.

Fala — Integrante 2:

> No sentido servidor para cliente, o iperf recebeu 94,4 megabits por segundo em A e 19 em B. O GET com um cliente e dez mebibytes ficou abaixo dessas referências, com 87,3 e 17,7. É uma comparação de contexto: o iperf mantém um fluxo por cerca de vinte segundos, enquanto o GET mede uma operação finita, incluindo conexão e processamento. Não atribuímos toda a diferença a um único custo. Com mais clientes, o aumento do tempo e a queda da taxa individual são compatíveis com disputa pelos recursos. Para localizar o ponto de saturação do serviço, precisamos medir o total concluído em uma janela comum. Somar taxas individuais ou usar a taxa do iperf não substitui essa medição da aplicação. O iperf funciona aqui como referência do caminho TCP e do limite aproximado da rede em um fluxo longo; ele não reproduz a abertura de uma conexão por arquivo, a leitura e gravação do conteúdo, nem a confirmação do protocolo da aplicação. A proximidade entre GET e iperf em um cliente mostra que a transferência individual utiliza boa parte da taxa observada no sentido servidor para cliente. Com 32 clientes, entretanto, o tempo individual cresce muito; sem a janela conjunta não afirmamos quanto o conjunto transferiu por segundo.

Slide 10 — Contribuições individuais

Adicionar ao slide: nomes reais e responsabilidades confirmadas, distribuídas em duas colunas:

Frente	[Nome 1] — aplicação	[Nome 2] — experimento e análise
Implementação	Protocolo, servidor e arquivos	Cliente, telemetria e coleta
Verificação	Operações, respostas de erro e concorrência	CORE, calibração, matriz e consistência das métricas
Resultados	Relação entre implementação e comportamento	Processamento em Python, estatística e gráficos
Atividades conjuntas	Integração, revisão cruzada e apresentação	Integração, revisão cruzada e apresentação

Fala — Integrante 1:

> Minha contribuição se concentrou no protocolo e no servidor, incluindo o atendimento concorrente e o tratamento dos arquivos. Também participei da integração com o cliente e da verificação das operações e respostas de erro. Na discussão dos resultados, ajudei a relacionar as decisões de implementação ao comportamento observado.

Fala — Integrante 2:

> Minha contribuição se concentrou no cliente, na telemetria e na organização dos experimentos no CORE. Trabalhei na calibração, na matriz de cenários e no processamento em Python, incluindo estatística e gráficos. Fizemos juntos a integração, a revisão dos resultados e a preparação da apresentação, compartilhando o entendimento das duas frentes.

Slide 11 — Autoavaliação e conclusão

Adicionar ao slide: três blocos curtos:

- **Avanços:** aplicação C concorrente; rede medida nos dois sentidos; 36 configurações analisadas.
- **Aprendizados:** rodada como unidade estatística; RTT e tempo da operação são diferentes; taxa individual e agregada também.
- **Evolução:** janela comum, integridade PUT, calibração repetida e monitoração de recursos.

Destacar “B: maior tempo em 18/18 comparações; menor taxa individual em 12/12 GET/PUT”. Rodapé com nomes e endereço do projeto, sem exibir páginas de referência.

Fala — Integrante 1:

> Nossa autoavaliação é positiva pela integração entre programação de rede e experimentação. Implementamos a aplicação em C, exploramos atendimento concorrente e analisamos uma matriz com diferentes cargas. As medições de rede nos dois sentidos tornaram a interpretação mais concreta: além dos parâmetros configurados, conseguimos observar taxa recebida, RTT e percurso. A divisão do trabalho ajudou a avançar na implementação e nos experimentos, enquanto a revisão conjunta ampliou o entendimento do sistema pelos dois integrantes. Um aprendizado importante foi distinguir sucesso informado pela operação de uma verificação completa do conteúdo recebido.

Fala — Integrante 2:

> O perfil B teve maior tempo nas dezoito comparações equivalentes e menor taxa individual nas doze comparações de GET e PUT. A concorrência aumentou o tempo de conclusão e reduziu a parcela de cada cliente. Como melhoria, priorizaríamos a janela comum para medir a taxa agregada, a conferência de integridade do PUT no servidor, mais ensaios independentes de calibração e o acompanhamento de CPU e disco. Também separaríamos banda e atraso em novos cenários para entender seus efeitos individuais. O trabalho consolidou nossa compreensão de como implementação, rede e carga influenciam a experiência de transferência. Obrigado.